

Spherepop on the Commodore

A Practical Introduction to PET and C64 Programming with VICE

Flyxion

August 2026

Abstract

This tutorial begins with a modern Ubuntu or WSL installation and ends with a tiny playable Commodore 64 game. Along the way we install VICE, launch Commodore PET and C64 emulators, enter short BASIC programs by hand, learn how to stop and clear programs, manipulate C64 screen memory directly, read held keys, fire a projectile, detect a collision, animate a small “pop,” and maintain a score. The final program is deliberately small enough to type into an emulated C64 while still exposing the architecture of a real game loop.

1 Why do this?

The Commodore machines are useful precisely because so little is hidden. A modern application may pass through an operating system, graphical toolkit, browser engine, virtual machine, driver stack, and GPU before a visible character appears. On a Commodore 64, a BASIC program can place a character on the display by writing a number directly into screen memory. The distance between an abstract rule and a physical-looking result is remarkably short.

This makes the machine a natural playground for Spherepop. We can begin with a few elementary state changes—position, creation, refusal at a boundary, collision, disappearance, and regeneration—and watch them become a game. The final loop can be read as a tiny operational system: move, fire, advance, compare, pop, score, regenerate, repeat.

2 Installing VICE on Ubuntu or WSL

VICE is the Versatile Commodore Emulator. Ubuntu packages it as `vice`; Ubuntu 24.04, for example, provides VICE 3.7.1 in the multiverse repository. The VICE manual identifies `xpet` as the PET emulator and `x64sc` as the accurate C64 emulator.

On Ubuntu or Ubuntu under WSL, begin with:

```
sudo apt update
sudo apt install vice
```

Check that the programs exist:

```
which xpet
which x64sc
x64sc -version
```

On WSL, graphical VICE windows require GUI support. Current WSL installations with WSLg normally provide this automatically. If a VICE window opens, the graphical side is already working.

Some packaged VICE installations do not contain every original Commodore ROM image for licensing or packaging reasons. A failure such as

```
C64MEM: Error - Couldn't load kernal ROM `kernal-901227-03.bin'.  
Error - Machine initialization failed.
```

means that the emulator itself was found but a required machine ROM was not. Supply legally obtained ROM images in the location expected by the installed VICE package or configure VICE to use the directory containing them. This is distinct from a BASIC programming error.

3 Trying the Commodore PET

Launch the PET from the shell:

```
xpet
```

After the familiar Commodore BASIC prompt appears, type a minimal program. On an emulator whose keyboard mapping makes shifted characters inconvenient, lowercase input is perfectly adequate:

```
10 print "hello from the pet"  
20 goto 10
```

Then type:

```
run
```

The PET will print the sentence indefinitely. Use the emulated **RUN/STOP** key to interrupt it. At the BASIC prompt,

```
list
```

shows the program currently stored in memory. A program can deliberately terminate with an **END** statement:

```
10 print "one pass through the world"  
20 end
```

A slightly more philosophical test is:

```
10 print "(the world (is all (that is the case)))"  
20 goto 10
```

The important lesson is not the text itself. We have already acquired the basic edit–run–interrupt–list cycle used throughout the rest of the tutorial.

4 Launching the Commodore 64

For the more accurate C64 emulation, run:

```
x64sc
```

VICE also provides `x64`, its faster C64 emulator, but `x64sc` is a good default when accuracy matters.

If VICE reports an error such as

```
Filesystem Image: Error - Cannot open directory `/some/path' as an image.  
Error - autostart disk attach failed.  
AUTOSTART: Error
```

it has been asked to autostart a path that it cannot attach. This is not a failure of the BASIC interpreter. Start `x64sc` without the bad autostart argument and work directly at the `READY.` prompt.

5 A few BASIC survival commands

The following commands are enough to recover from most early experiments. `RUN` executes the program. `LIST` displays it. `END` terminates execution when encountered. The physical or emulated `RUN/STOP` key interrupts a running program.

To clear the C64 screen from BASIC, use:

```
print chr$(147)
```

Entering a line number by itself deletes that program line. Thus

```
200
```

deletes line 200. Typing a numbered line again replaces the old version, which makes incremental patching convenient.

One historical peculiarity matters enormously: Commodore BASIC V2 distinguishes variable names using only their first two significant characters. Long descriptive names therefore collide surprisingly easily. In this tutorial we deliberately use short names such as `sx`, `sy`, `sc`, `bx`, and `ox`.

6 The first movable object

Begin with the smallest possible player. Type:

```
10 x=20  
20 print chr$(147)  
30 print tab(x);"^"  
40 get a$  
50 if a$="a" and x>1 then x=x-1  
60 if a$="d" and x<38 then x=x+1  
70 goto 20
```

After RUN, pressing A moves the arrow left and D moves it right. This version clears and redraws the entire screen repeatedly. It is crude, but it proves that input, state, rendering, and looping all work.

Conceptually, x is the state of the player. The two IF statements also implement a primitive refusal rule: an attempted movement beyond the chosen boundary is simply not admitted.

7 Why repeated PRINT eventually becomes a problem

A first attempt at adding bullets can use many PRINT statements to position objects vertically. Unfortunately, repeatedly printing newlines eventually reaches the bottom of the C64 display and causes the screen to scroll. What looks like random graphical corruption may simply be the terminal behavior of BASIC doing exactly what it was told.

The better solution is to stop treating the display as a scrolling terminal and treat it as memory.

8 C64 screen memory

In the normal C64 text configuration, screen memory begins at address 1024 and contains 40 columns by 25 rows. A useful address calculation is therefore

$$1024 + \text{row} * 40 + \text{column}$$

A BASIC POKE writes a byte to an address. For example,

```
poke 1024+22*40+x,30
```

writes screen code 30 into row 22 at horizontal position x. Writing screen code 32 places a blank and is therefore a convenient way to erase an old object.

This is the conceptual turning point of the tutorial. We are no longer asking BASIC to print a page. We are maintaining a little world represented directly in display memory.

9 Held-key movement

GET A\$ consumes characters from BASIC's input mechanism, so movement can feel like repeated tapping rather than game control. The C64 also exposes the currently detected key at memory location 197. We can read it with:

```
k=peek(197)
```

For the keyboard mapping used here, A is 10, D is 18, and space is 60. Thus:

```
60 k=peek(197)
70 if k=10 and x>0 then x=x-1
80 if k=18 and x<39 then x=x+1
90 if k=60 and b=0 then b=21:c=x
```

Holding A or D now produces continuous movement. This simple method is essentially single-key input; a later machine-language or keyboard-matrix version can recognize combinations such as movement and fire simultaneously.

10 Spherepop Shooter

We can now assemble the complete BASIC prototype. The player is a character near the bottom of the screen. A sphere appears near the top. Space creates one projectile. The projectile advances upward. A collision causes a short character animation, increments the score, erases the sphere, and generates another sphere at a random horizontal coordinate.

The program intentionally remains a character-mode prototype. It is small enough to type by hand and transparent enough that every state transition can be inspected.

10.1 Complete program

Start from an empty BASIC program if desired with `NEW`, then enter the following. The code uses direct screen memory and held-key input.

```
10 x=20:b=0:ox=x:p=0:sc=0
20 sx=10:sy=2
30 print chr$(147)
40 poke 1024+sy*40+sx,81
45 print chr$(19);"pops:";sc
50 poke 1024+22*40+x,30
60 k=peek(197)
70 if k=10 and x>0 then x=x-1
80 if k=18 and x<39 then x=x+1
90 if k=60 and b=0 then b=21:c=x
100 poke 1024+22*40+ox,32
110 poke 1024+22*40+x,30
120 ox=x
130 if p>0 then poke 1024+p*40+c,32
140 if b=0 then 210
150 b=b-1
160 if b=sy and c=sx then 220
170 if b<1 then b=0:p=0:goto 60
180 poke 1024+b*40+c,93:p=b
190 goto 60
210 goto 60
220 poke 1024+sy*40+sx,42
225 for d=1 to 40:next
230 poke 1024+sy*40+sx,43
235 for d=1 to 40:next
240 poke 1024+sy*40+sx,32
245 sc=sc+1
250 print chr$(19);"pops:";sc
255 b=0:p=0
260 sx=int(rnd(1)*38)+1
265 poke 1024+sy*40+sx,81
270 goto 60
```

Type **RUN**. Hold A and D to move. Press space to fire. Line up with the sphere and hit it. The sphere changes characters briefly, disappears, the **POPS** count increases, and another sphere appears.

11 Reading the game as a state machine

The program is more interesting than its graphics suggest. The player has a horizontal state x . The target has a position (sx, sy) . The bullet has a vertical state b and a captured horizontal coordinate c . Firing binds the projectile to the player's horizontal coordinate at the instant of creation. Boundary tests refuse player transitions outside the screen. Each loop advances the projectile. Line 160 asks whether projectile and target occupy the same cell. If they do, control transfers to the pop routine.

The crucial transition is:

```
160 if b=sy and c=sx then 220
```

The collision is therefore not a graphical illusion. It is an equality relation over two pieces of state. Lines 220–240 make that state transition visible as a tiny explosion. Line 245 records the event:

```
245 sc=sc+1
```

Finally, lines 260–265 create a new admissible target state. The entire game is a repeated transformation of a very small state space.

12 Debugging lessons from building it interactively

Several failures encountered while constructing the game are worth preserving because they reveal the machine. A screen that appears to move unpredictably may be ordinary scrolling caused by too many `PRINT` operations. A syntax error reported at a particular line should first be inspected with `LIST 190`, or the corresponding line number, because the stored program is authoritative: the character that actually reached the emulator may differ from what was intended. Re-entering a numbered line replaces it, while entering only its number deletes it.

Short variable names are not merely stylistic nostalgia. Because BASIC V2 recognizes only the first two significant characters of variable names, names that appear distinct to a modern programmer may denote the same variable. Restricting the program to compact one- and two-character names avoids an entire class of mysterious errors.

VICE errors should likewise be separated from BASIC errors. A missing KERNAL ROM occurs before the emulated machine can initialize. An autostart filesystem error concerns a host path VICE attempted to attach. A `?SYNTAX ERROR IN 190` comes from the Commodore BASIC program itself. Keeping those layers distinct makes debugging much easier.

13 Where to take Spherepop next

The present program is a proof of concept rather than the endpoint. The most natural next step is to replace character graphics with the C64's hardware sprites. A sphere can then become a genuinely circular multicolor object and the player can become a small ship. Direct keyboard-matrix scanning would permit simultaneous movement and firing. Multiple projectiles and multiple spheres would turn the single collision relation into a collection of interacting objects. Color RAM could make each pop flash independently. SID sound registers could turn the pop into an audible event.

Beyond BASIC, the same design can be rewritten in 6502 assembly. That would not change the conceptual game: player state, projectile state, target state, collision, pop, score, regeneration. It would instead change how directly and rapidly those operations are expressed. The BASIC prototype therefore serves as an executable specification for a later assembly version.

The larger lesson is that expressive complexity need not begin with a large primitive vocabulary. A few bytes of state, a few comparisons, and a repeated transition loop are enough to produce recognizable agency, interaction, consequence, memory, and recurrence. On a C64, those abstractions remain close enough to the machine that one can watch them become pixels—or, for now, characters—almost one operation at a time.

14 Spherepop with hardware sprites, multiple targets, and sound

The character-mode prototype proves the design but hides three things a real C64 game normally exploits: sprites, a hardware collision test, and the SID sound chip. This section rebuilds the same state machine (player, projectile, targets, collision, pop, score) on top of those three facilities. The state transitions do not change. What changes is which piece of hardware notices and announces them.

14.1 Sprites replace characters

A VIC-II sprite is a 24-by-21-pixel bitmap: three bytes per row, twenty-one rows, sixty-three bytes total. Sprite data is stored in 64-byte-aligned blocks, so a block number is simply the starting address divided by 64. Block 13 begins at address 832; block 14 begins at address 896. Each sprite has a one-byte pointer, located just past the end of screen memory. With the screen at 1024, the eight sprite pointers occupy addresses 2040 through 2047: sprite 0's pointer is at 2040, sprite 1's at 2041, and so on.

Writing the sprite bitmap therefore takes two steps: place the sixty-three bytes somewhere convenient, then tell the corresponding pointer which 64-byte block they start in.

```
20 for i=0 to 62:read a:poke 832+i,a:next i
30 for i=0 to 62:read a:poke 896+i,a:next i
40 poke 2040,13
50 poke 2041,14
60 poke 2042,14:poke 2043,14:poke 2044,14
```

Sprite 0 (block 13) becomes the player. Sprites 1 through 4 (block 14) share a single circular shape: sprite 1 is the bullet, sprites 2 through 4 are three simultaneous targets. Nothing prevents several sprites from pointing at the same 64-byte block; the VIC-II only cares where each sprite's pointer sends it, not whether other sprites are sent there too.

The bitmaps themselves come from DATA statements read once at program start. BASIC's READ draws from DATA statements in ascending line-number order regardless of where in the program the READ happens to execute, so it is idiomatic to place the data at the very end of the listing, as shown below. The exact pattern of ones and zeros is not conceptually important; what matters is that a sprite, unlike a character cell, is addressed once, as an object, rather than one screen cell at a time.

14.2 Registers replace scattered POKEs

Every sprite property has a fixed offset from the VIC-II base address, 53248. If $v=53248$, then $v+0$ and $v+1$ are sprite 0's X and Y position, $v+2$ and $v+3$ are sprite 1's, and so on up to sprite 7. Enabling sprites is a single write to $v+21$ (53269), one bit per sprite: a value of 31 (binary 00011111) turns on sprites 0 through 4 and leaves 5, 6, and 7 dark. Sprite colors live at $v+39$ through $v+46$.

```
70 poke 53269,31
80 poke v+39,1:poke v+40,7:poke v+41,2:poke v+42,5:poke v+43,6
```

An inactive projectile or target is not erased; it is parked off-screen, conventionally by giving it a Y coordinate of 255. This is the same refusal idea seen earlier in the tutorial, applied to visibility rather than movement: a sprite is simply not admitted into the visible region until its state calls for it.

14.3 Three targets, one array

The character-mode version tracked a single target with two scalars, sx and sy . With three simultaneous targets, scalars become arrays:

```
130 dim tx(3),ty(3)
140 tx(1)=60:ty(1)=60
150 tx(2)=140:ty(2)=60
160 tx(3)=220:ty(3)=60
170 for i=1 to 3:poke v+2+i*2,tx(i):poke v+3+i*2,ty(i):next i
```

Target i occupies sprite $(i+1)$, so its position registers are $v+2+i*2$ and $v+3+i*2$. The state space has grown from one target's position to three, but the update rule is the same rule applied three times through a loop rather than three separately named variables.

14.4 The collision register replaces the equality test

The earlier program tested collision with a single line, `if b=sy and c=sx then 220`: an explicit equality between two pieces of software state. The VIC-II offers a hardware equivalent. Address 53278 is the sprite-to-sprite collision register. Whenever two or more sprites' opaque pixels overlap on screen, the VIC-II sets one bit per involved sprite in this register and leaves it set until the CPU reads it, at which point it clears automatically. A single PEEK therefore reports every sprite involved in a collision since the last check, with no coordinate arithmetic at all.

```
400 cc=peek(53278)
410 if (cc and 2)=0 then 500
420 for i=1 to 3
430 if (cc and (2^(i+1)))=0 then 460
440 gosub 600
450 ba=0:by=255
460 next i
```

Bit 1 (value 2) belongs to the bullet, sprite 1. Bits 2 through 4 (values 4, 8, and 16) belong to targets 1 through 3, sprite 2 through 4. Line 410 first asks the cheap question, whether the bullet was involved in anything at all, before line 420 asks the more expensive one, which target specifically. This mirrors ordinary refusal logic: a fast, coarse admissibility check gates a slower, more specific

one. The trade is that the hardware register reports *any* overlap among enabled sprites, including an incidental brush between the player and a target; the character-mode version's explicit equality test could never be confused this way. A production version would keep the player on a sprite that starts disabled, or would filter the register more carefully, but for a hand-typed prototype the simplification is worth the brevity.

14.5 Sound as a fifth state transition

The original program marked a hit with two character changes and a pair of empty timing loops. The SID chip lets the same event become audible. Three registers matter for a short, percussive “pop”: address 54296 is the master volume, and must be nonzero or every subsequent sound is silent; 54277 packs voice 1's attack and decay into one byte; 54276 is voice 1's control register, whose bits select waveform and, in its lowest bit, gate the note on or off.

```
190 poke 54296,15
200 poke 54277,9:poke 54278,0:poke 54273,20
```

```
600 poke 54276,129:for d=1 to 20:next d:poke 54276,128
610 sc=sc+1
620 tx(i)=int(rnd(1)*260)+30
630 ty(i)=int(rnd(1)*100)+40
640 poke v+2+i*2,tx(i):poke v+3+i*2,ty(i)
650 return
```

The value 129 written to the control register selects the noise waveform (128) and sets the gate bit (1) in the same write, starting the note; 128 alone releases it a few dozen instructions later. Line 245's silent `sc=sc+1` from the character-mode version is now bracketed by an audible event on one side. The pop subroutine ends by choosing a fresh, random position for whichever target was hit and writing it directly to that target's position registers, the same “generate another sphere” step as before, now applied to one array slot instead of the only target that existed.

14.6 Complete program

```
10 v=53248
20 for i=0 to 62:read a:poke 832+i,a:next i
30 for i=0 to 62:read a:poke 896+i,a:next i
40 poke 2040,13
50 poke 2041,14
60 poke 2042,14:poke 2043,14:poke 2044,14
70 poke 53269,31
80 poke v+39,1:poke v+40,7:poke v+41,2:poke v+42,5:poke v+43,6
90 x=100:yp=220
100 poke v+0,x:poke v+1,yp
110 bx=0:by=255:ba=0
120 poke v+2,bx:poke v+3,by
130 dim tx(3),ty(3)
140 tx(1)=60:ty(1)=60
150 tx(2)=140:ty(2)=60
160 tx(3)=220:ty(3)=60
170 for i=1 to 3:poke v+2+i*2,tx(i):poke v+3+i*2,ty(i):next i
180 sc=0
```

```

190 poke 54296,15
200 poke 54277,9:poke 54278,0:poke 54273,20
300 print chr$(19);"pops:";sc
310 k=peek(197)
320 if k=10 and x>24 then x=x-3
330 if k=18 and x<300 then x=x+3
340 poke v+0,x
350 if k=60 and ba=0 then ba=1:bx=x:by=yp-10
360 if ba=0 then 400
370 by=by-6
380 if by<50 then ba=0:by=255
390 poke v+2,bx:poke v+3,by
400 cc=peek(53278)
410 if (cc and 2)=0 then 500
420 for i=1 to 3
430 if (cc and (2^(i+1)))=0 then 460
440 gosub 600
450 ba=0:by=255
460 next i
500 goto 300
600 poke 54276,129:for d=1 to 20:next d:poke 54276,128
610 sc=sc+1
620 tx(i)=int(rnd(1)*260)+30
630 ty(i)=int(rnd(1)*100)+40
640 poke v+2+i*2,tx(i):poke v+3+i*2,ty(i)
650 return
9000 data 0,28,0,0,28,0,0,62,0,0,62,0
9010 data 0,127,0,0,255,128,0,255,128,1,255,192
9020 data 1,255,192,3,255,224,7,255,240,7,255,240
9030 data 15,255,248,15,255,248,31,255,252,63,255,254
9040 data 127,255,255,255,255,255,12,0,24,12,0,24
9050 data 12,0,24
9100 data 0,0,0,0,126,0,1,255,128,3,255,192
9110 data 7,255,224,15,255,240,31,255,248,31,255,248
9120 data 31,255,248,31,255,248,63,255,252,31,255,248
9130 data 31,255,248,31,255,248,31,255,248,15,255,240
9140 data 7,255,224,3,255,192,1,255,128,0,126,0
9150 data 0,0,0

```

Type RUN. Hold A and D to slide the ship; hold space to fire. Three targets are visible at once instead of one, each an independently addressable sprite rather than a screen character, and each pop is heard as well as seen. Lines 9000 through 9150 are the only lines that changed in *kind* rather than degree: they are not instructions but a resource, a small bitmap the rest of the program treats as a fixed asset, loaded once and referenced by pointer for the remainder of the run. That distinction between instruction and asset is itself worth noticing, since it is the same distinction that separates a program from the sprites, sounds, and levels a larger game would eventually load from disk.

14.7 What this version still refuses

The bullet parks at by=255 and the player sprite is always enabled, so a bullet and the player sprite could in principle register a collision if their bounding boxes ever touched; the program does not defend against that case, and a careful reader may want to add it as an exercise. Sprite positions are

also kept under 255 to avoid the VIC-II's ninth-bit X-coordinate register at 53264, which restricts the player to the left three-quarters of the screen. A further exercise is to read and set that register so the whole 320-pixel width is reachable, and to give the targets independent vertical drift so they no longer sit on a single fixed row.

References and further reading

VICE Team. *VICE Manual*. In particular, the emulator invocation documentation identifies `x64`, `x64sc`, and `xpet`. Available from <https://vice-emu.sourceforge.io/>.

Ubuntu Packages. *vice* — *Versatile Commodore Emulator*. Ubuntu package archive, including the Ubuntu 24.04/Noble VICE package. Available from <https://packages.ubuntu.com/noble/vice>.